Script as Brain, LLM as Eyes: An End-to-End GUI Testing Harness Spanning Desktop, Dual-Browser Web, Audio, and a Production Cloud

Mahmudul Faisal Al Ameen

Draft — July 2026 · developed and operated with an LLM coding assistant (Claude)

Abstract. GUI test automation traditionally binds tests to selectors or accessibility identifiers — brittle under UI change and unavailable on surfaces the tester does not control (native file dialogs, browser chrome, foreign applications). Autonomous LLM "computer-use" agents remove the selector dependency but sacrifice determinism: two runs of an improvising agent are not the same test. We report on a harness that splits the difference: **control flow is a deterministic, prewritten script; a vision LLM is used exclusively as the perception organ** — locating elements from natural-language descriptions, judging screen states, and reading dynamically generated values off pixels. Perceived state is never the final arbiter: every scenario terminates in hard, non-visual oracles — SQLite inspection, public HTTP endpoints of the production backend, perceptual image hashing against source documents, and audio waveform analysis. The harness drives, with one step vocabulary, a native PyQt desktop application (including a GTK file dialog), two different real browsers concurrently under distinct user accounts, a synthetic presenter, an OS-level virtual microphone with a closed audio loop through a WebRTC mesh, fault-injected networking, and a live production cloud. Applied to SeenSlide, a real slide-sharing product, 18 scenarios found **ten production bugs** — none of which the project's 65-test unit suite could have caught — including silent data loss to volatile storage, a dead database pool 500-ing an API for every authenticated user, and a three-bug cascade in which each fix exposed the next. Across roughly thirty runs the vision layer caused zero state-damaging misclicks. We describe the architecture, the methodology rules learned from failures, the full bug inventory, and the limits of the approach.

1 Introduction

End-to-end GUI testing occupies an awkward position in practice. It is the only test layer that exercises a product the way users do, yet it is the layer teams most often abandon: selector-bound scripts decay with every UI change; cross-platform products need parallel harnesses per surface; and whole classes of behavior — audio, multi-user synchronization, degraded networks, the real production backend — are usually declared out of scope.

Two recent developments change the available design space. First, vision-language models (VLMs) can now locate UI elements from natural-language descriptions with production-grade reliability, an ability commercialized by several testing vendors and benchmarked by the research community [1] [6]. Second, "computer-use" agents demonstrate that an LLM can operate arbitrary GUIs through screenshots, mouse, and keyboard. The dominant research direction combines both into **autonomous testing agents** that explore an app and improvise test actions [2] [4]. Autonomy, however, is a poor fit for regression testing: an improvising agent does not run the same test twice, cannot be reviewed as a test artifact, and its failures are hard to attribute.

This paper reports on a different point in the design space, built for and evaluated on SeenSlide — a real, deployed product comprising a PyQt5 desktop application for slide capture, a FastAPI/PostgreSQL cloud backend, and a Svelte web viewer with real-time collaboration. The harness rests on three principles:

1. **Deterministic control flow.** Scenarios are prewritten YAML step lists. A run either follows the script or fails. Tests are reviewable, diffable artifacts.
2. **LLM as perception only.** The vision model answers exactly three kinds of questions: *where is the element matching this description?* (`locate`), *does the screen show X?* (`judge`), and *what value is displayed here?* (`read`). It never chooses what to do next.
3. **Hard oracles above perception.** Visual judgment is a means, not the verdict. Scenarios end in database queries, public API checks, perceptual hashes against source PDFs, and audio waveform assertions.

The resulting system is, to our knowledge, the first published harness in which mouse, keyboard, vision, **closed-loop audio**, network fault injection, and production-cloud verification operate together across a desktop application and two concurrent browsers (Section 2 discusses the closest systems). Its practical yield on a real product — ten production bugs found, fixed, and shipped within the same effort — is the paper's central evidence.

Contributions.

- An architecture separating deterministic test scripts from LLM perception, with a three-primitive perception interface (locate / judge / read) and a coordinate cache enabling zero-LLM replays (Section 3).
- Techniques for dimensions GUI testing normally excludes: an OS-level virtual microphone driving a real WebRTC mesh with waveform-verified output, including proofs of **silence**; a synthetic fullscreen presenter as the "human" under test; an HTTP relay as a network fault injector; window-scoped actors enabling two real browsers with different logins as first-class test participants (Section 3.3).
- An oracle taxonomy pairing every visually-judged behavior with a non-visual ground truth (Section 3.4).
- An experience report on a production system: 18 scenarios, ten bugs with root causes, a quantified reliability and cost profile, and methodology rules distilled from real failures (Section 5, Section 6).

2 Related Work

LLM-driven GUI testing. Most work targets mobile apps with autonomous or semi-autonomous agents: scenario-guided generation [2], Android task automation [3], and multi-agent exploration. These systems **generate** actions; ours executes fixed scripts and delegates only perception. Desktop GUIs are a recognized gap — the UI-Vision benchmark [1] notes that existing agent benchmarks focus on web and mobile.

Commercial AI test automation. Vendors offer natural-language test authoring, self-healing selectors, and VLM-based visual testing [6] [7] [8]. These are predominantly web/mobile products; audio, native dialogs, and cross-application flows are out of scope. Our locate primitive is philosophically similar to "AI locators", applied uniformly to surfaces such tools do not reach.

Multi-user feature testing. Recent work coordinates multiple LLM agents on separate devices to trigger multi-user features in mobile apps [4] [5]. We test two-user collaboration deterministically: two named browser actors in one script, with the LLM reading a dynamically generated invite code off one actor's screen into a variable typed into the other.

Voice and WebRTC testing. A specialized industry tests voice agents and WebRTC services [9] [10] [11], typically via fake media streams or telephony rigs, evaluating STT/LLM/TTS pipelines. We found no published system that closes the audio loop at the **operating system** level (TTS → virtual microphone → application → WebRTC → speaker sink → recorded waveform) as one oracle inside a general GUI harness.

3 Design

3.1 Architecture

```
scenario.yaml ─▶ runner ─▶ locator ─▶ vision LLM (claude-opus-4.8)
                   │           └── coordinate cache ( --replay ⇒ 0 LLM calls)
                   ├─▶ actions    xdotool click/type/key/drag · mss screenshots
                   │              browser ACTORS (window-cropped, offset clicks)
                   │              sandboxed app lifecycle
                   ├─▶ presenter  PDF → fullscreen Qt window ("the human presenting")
                   ├─▶ audio      piper-TTS + pw-loopback virtual microphone
                   ├─▶ relay      local HTTP relay → cloud (kill = network outage)
                   └─▶ reports    film-strip HTML + JSON + logs + DBs + WAVs
```

Listing 1: Harness components. The LLM appears in exactly one box.

The **runner** interprets a YAML step list. The step vocabulary covers lifecycle (`launch`, `kill`, `actors`), input (`click`, `type`, `key`, `draw`), perception (`assert_screen`, `read_text`), environment (`present_pdf`, `virtual_mic`, `play_audio`, `relay`), and oracles (ten `verify_*` steps, Section 3.4). Values read from the screen become `$variables` usable in later `type` and `assert_screen` steps.

The **locator** wraps the vision model behind three primitives. `locate` returns a bounding box and click point for a natural-language description; `judge` returns a boolean with a one-sentence reason; `read` returns a displayed value verbatim (never cached — values change per run). Locate results are cached by (screen size, description); a replay mode re-runs entire scenarios from the cache with zero model calls. A mid-tier vision model suffices: localizing labeled controls on clean UIs does not require frontier capability, and the mid-tier model is roughly twice as fast.

Input synthesis uses OS-level events (xdotool on X11), which are indistinguishable from a human — important here because the application under test triggers screen captures from an input monitor. Screenshots come from `mss`; on multi-monitor setups the primary is resolved via xrandr and click coordinates are offset-corrected.

3.2 Sandboxing the application under test

The desktop app runs under a disposable `$HOME` (isolated settings, credentials, database), a null keyring backend — during development the sandbox querying the real Secret Service popped the operator's password manager mid-run — and, for cloud scenarios, a **pre-seeded random device identifier**. The latter is a safety property: the app's fallback derives identity from `/etc/machine-id`, so an unseeded sandbox would silently resurrect and pollute the developer's real cloud account. Everything a cloud run creates belongs to a throwaway anonymous identity that is purged afterwards; purges are name-gated inside the SQL itself (patterns like `@harness.invalid`), never lists of identifiers collected elsewhere (Section 6 motivates this rule).

3.3 Testing dimensions beyond point-and-click

The presenter. Slide-capture products are exercised by someone presenting. A small Qt program renders PDF pages fullscreen on a chosen monitor under a schedule (first-page hold, per-page interval, page ranges for multi-segment talks). Two compositor lessons: the window must be bound to its target `QScreen` and shown with `showFullScreen()` — frameless windows placed by `setGeometry` are silently relocated by Mutter on multi-monitor — and the application under test needs an own stay-on-top hint, because the window manager ignores external raise requests.

Closed-loop audio. A text-to-speech engine (piper) speaks one line per slide, silence-padded to the exact presentation schedule. A PipeWire loopback pair (a sink whose playback side is a **device-class** source) becomes the system default microphone for the run and is restored afterwards; this was the only recipe that works — `pw-cli create-node` silently creates nothing and the session manager refuses stream-class nodes as defaults. On the output side, the harness records the default sink's monitor and asserts waveform peaks.

The combination yields strong claims: in the WebRTC voice-chat test, the TTS feeds **only** the loopback microphone, so any energy on the speaker sink can only have crossed the mesh from the sending browser (measured peak 1.0; a broken transmit path measures 0.000). A `max_peak` mode proves **silence** — e.g. that pausing playback actually stops audio.

Network fault injection. Cloud scenarios route the app through a local HTTP relay; killing the relay is a clean outage (instant connection-refused), restarting it restores service — no host network changes, no privileges. One integration subtlety: the relay must strip `Content-Encoding` because the forwarding client already decodes bodies.

Browser actors. A step `actors: {A: edge, B: firefox}` binds names to real, user-prepared browser windows by title and browser marker; the largest match wins and is self-verified by a `judge` call at registration. Steps carrying `target: A|B` crop the screenshot to the actor's current window rectangle (re-queried per step; clamped to the virtual screen, since the compositor parks windows a few pixels offscreen) and offset clicks into it. Two viewers of the **same page** therefore cannot be confused, and the harness physically cannot click outside an actor's window — relevant because the operator's own terminal shared a monitor with one actor throughout the evaluation.

3.4 Oracles

Oracle	Ground truth established
`verify_db`	slide counts and per-talk 1..N sequence contiguity in the app's SQLite
`verify_slides_match_pdf`	every stored slide perceptually matches some presented source page (best-match dhash — deduplication legitimately merges near-identical pages, so positional matching is wrong) and matched pages appear in presentation order
`verify_cloud`	the **public** session endpoint — what the audience's viewer renders — reports expected slide and viewer counts
`verify_talks`	a conference deck was split into the scheduled talks; each talk's first **visible** slide equals its title page
`verify_voice`	the app's own log line plus the recorded WAV's duration and peak
`verify_hidden`	the slide-quality gate armed; hidden counts bounded; **cloud == kept exactly** — an upload leak and a failed re-upload both fail
`verify_outbox`	pending upload-retry rows (then zero after recovery)
`verify_audio_out`	waveform peak on the system output — or, inverted, proof of silence
`verify_log`	regex evidence that a specific internal action ran

Table 1: The oracle layer. Every visually-judged behavior is paired with a non-visual check.

The oracle layer is what makes LLM judgment safe to rely on: a screen that merely **looks** right cannot pass a scenario. Three of the ten bugs (Section 5.1) were invisible on screen and caught purely by oracles.

4 Scenarios

Eighteen scenarios were written against SeenSlide. Desktop (P0–P5): sandboxed smoke navigation (verified on one- and two-monitor setups); a complete talk from form to captured, deduplicated, stored slides, perceptually verified against the source PDF; the same through the real cloud under a throwaway identity, verified on the public endpoint; a narrated talk through the virtual microphone with slide markers; a live cloud talk watchable by a human mid-run; conference mode — a CSV schedule imported through the **native GTK file dialog** (via its Ctrl+L location bar), then a synthetic three-talk deck split entirely by OCR-based title auto-advance with zero manual clicks, each talk's first slide equal to its title page at hash distance 0; the same split by manual "Next Talk" clicks between presenter segments; the hidden-slide recovery flow ("Is it a slide?

Unhide"), verified two-sidedly (flag cleared **and** the late upload happened); and an offline scenario in which the network dies mid-talk, slides queue, and a background worker heals the audience's deck on recovery.

Web (W0–W6), with Edge and Firefox as actors A and B under different accounts: independent navigation; live following (a late joiner lands on the **current** slide; auto-advance under a LIVE badge; live audio after the browser-mandated "listen" gesture; presence-counted viewers; the end-of-talk transition — while actor A, hidden beneath the fullscreen presenter for the whole talk, is verified afterwards to have tracked it blind); group collaboration (create; the LLM **reads** the generated 8-character join code off A and the script types it into B; chat both directions with database confirmation); private whispers; shared ink rendering live across browsers; WebRTC voice with the audio-loop proof above; recorded replay (audible playback, silence on pause, a deterministic marker walk performed **while paused**); AI features (a region explanation and follow-up from the production LLM backend, per-user isolation, and in-place slide translation rendered in French in one browser while the other keeps the original); and authentication — cookie persistence on the real accounts, plus a full HTTP-level security suite (single-use magic-link tokens, session invalidation on logout, identity separation) using self-purging throwaway accounts whose email domain (`.invalid`, RFC 2606) can never receive mail.

5 Evaluation: experience on a production system

The harness was built and applied over roughly four days in July 2026, alongside (and gated by) the project's conventional 65-test pytest suite. All scenarios ended green; the findings below are what it took to get there.

5.1 Bugs found

#	Found by	Defect	Impact
1	P1 oracle	`storage:` config section ignored; the shipped key (`base_dir`) was read by nothing	all local slide data in volatile `/tmp` — lost on every reboot, on every real install
2	P2 `verify_voice`	slide-marker wiring existed only in the cloud branch of voice recording	local recordings permanently unsyncable to slides
3	P3 visual	hardcoded white input background under a dark-theme white text token	imported schedules invisible
4	P3 + user report	slide-quality gate armed at countdown **end** (deck already covers the desktop) and exempted frames whose focused window was the app itself — exactly the post-talk junk	application screenshots uploaded into the audience's deck
5	P3 `verify_cloud`	a session slide counter incremented on upload was decremented by **no** delete path	public API over-reported counts
6	P4 flow	session row never persisted; the in-memory/database divergence was masked by an equality guard	the Sessions view showed "no talks" for every collection-based talk
7	W1 `verify_cloud`	`update_viewer_count` had zero callers (viewers poll REST; there is no socket to count)	viewer counts always 0
8	W6 probe	four backend modules each instantiated a private, never-connected DB pool	`/api/credits/balance` returned 500 for **every** authenticated user
9	exposed by #8	`db.transaction()` called at five sites — the method did not exist, and inner queries used separate pooled connections regardless	credit grant/spend, referrals, key spend, account merge: erroring and non-atomic
10	exposed by #9	welcome bonus granted twice once #9 made the second grant path work	new accounts credited 40 instead of 20

Table 2: Ten production bugs found by the harness. All were fixed and deployed during the same effort; the unit-testable ones received same-day regression tests.

Three observations follow. First, **every bug is an integration bug**: config plumbing, event-wiring branches, theme×widget interaction, compositor timing, cross-repository counter bookkeeping, memory-versus- database divergence, and dead wiring that type-checks. None was reachable by the unit suite, whose 65 tests were green throughout. Second, **fix cascades are real**: bugs 8→9→10 form a chain in which each repair exposed the next — only end-to-end re-verification after each fix (live probes with throwaway accounts) caught the follow-ons. Third, **oracles beat eyes**: bugs 2, 5, and 7 produced screens that looked perfectly normal.

5.2 Reliability and cost

Across roughly thirty scenario runs: **zero** state-damaging misclicks attributable to the vision layer. Failed runs were overwhelmingly correct rejections — a real product bug, real state drift from an earlier failed run, or a wrong assertion. We observed one vision-flake pattern (a refusal to locate a plainly visible element, three retries in a row) roughly once per several hundred calls; a plain re-run passed, and re-run-once became policy before treating a locate-miss as a regression.

Scenarios issue 7–25 model calls (5–15 s each on the CLI backend) and run one to five minutes wall-clock. The coordinate cache replays click/type flows with zero calls. Notably, several scenario "failures" during

development were **oracle** bugs, not product bugs — a deck whose fourth page legitimately is a title slide; a voice bar that has no speaking indicator to assert on; an auth probe aimed at an endpoint with an unrelated defect (which turned out to be bug #8). The discipline that emerged: read the run artifacts before touching product code.

6 Methodology rules learned from failures

- **State tolerance.** Runs die mid-way and the next run inherits the leftovers. Steps that open, join, or play must be optional or leave-then-redo. The sharpest instance: a WebRTC call surviving from a previous run holds a **dead** virtual-microphone node and transmits silence — every voice scenario now leaves and rejoins within the run that owns the microphone.
- **Never interact during continuous playback.** Perception latency is wall-clock time; a marker-walk test drifted nondeterministically as audio played under the locator's thinking. Pause first, then walk.
- **Two-sided oracles.** "Cloud may not exceed kept" passes when an upload silently fails; the unhide test requires exact equality so that both leak directions fail.
- **Cleanup discipline.** Delete only data whose test origin is provable in the delete statement itself. This rule exists because a session id collected from a server log was wrongly presumed test data and deleted; it was reconstructed from surviving storage-volume files, and the near-miss hardened the policy. Corollary: server-side caches can keep serving DB-deleted entities until a restart.
- **Autoplay policies are features, not bugs.** Browsers gate audio on a user gesture; scenarios click "listen" rather than reporting silence.
- **Blind actors are still actors.** When the fullscreen presenter covers a browser, that browser still follows the talk; scripts verify it after the deck ends rather than forfeiting the monitor.

7 Limitations

The harness is Linux/X11-specific in its input and capture layers; Windows, macOS, and Wayland remain future work. Runs require exclusive use of the screen. Perception latency (seconds per call) rules out tests of sub-second interactive behavior. Natural-language assertions can pass for the wrong reason — the standing mitigation is the hard- oracle layer, which every scenario must end with. The evaluation is a single product and a single operator; the bug yield, while concrete, is an existence proof rather than a controlled comparison. Finally, the locator's occasional refusals mean flake policy is part of the methodology, not an afterthought.

8 Conclusion

Treating a vision LLM as **only** the eyes of an otherwise deterministic test script turns end-to-end GUI testing into something a small project can actually afford — including the dimensions that are usually declared untestable: native dialogs, two browsers with two identities, real audio through a real WebRTC mesh, dying networks, and the production backend. On a real product the approach paid for itself immediately: ten shipped fixes, three of them for bugs no screenshot would ever show. The ingredients are individually known; their integration, and in particular the closed OS-level audio loop inside a general GUI harness, appears to be new. We offer the architecture and the methodology rules as a reusable recipe.

Acknowledgment. The harness, scenarios, and this draft were co-developed with an LLM coding assistant (Anthropic Claude), which also served as the harness's vision model operator during evaluation.

References

1. UI-Vision: A Desktop-centric GUI Benchmark for Visual Perception and Interaction. arXiv:2503.15661.
2. Scenario-Guided LLM-based (Mobile App) GUI Testing / ScenGen. arXiv:2506.05079.
3. LELANTE: Leveraging LLM for Automated Android Testing. arXiv:2504.20896.

4. Agent for User: Testing Multi-User Interactive Features in TikTok. arXiv:2504.15474.
5. Breaking Single-Tester Limits: Multi-Agent LLMs for Multi-User Feature Testing. arXiv:2506.17539.
6. QA Wolf. The 12 Best AI Testing Tools in 2026. qawolf.com/blog.
7. TestMu AI (LambdaTest). LLM UI Testing: Smarter Interface Testing with AI. testmuai.com.
8. Drizz. Vision Language Models: The Next Frontier in AI-Powered Mobile App Testing. drizz.dev.
9. Hamming AI. How to Test Voice Agents Built with LiveKit. hamming.ai/blog.
10. Cekura. Testing Pipecat Voice Agents: Simulation, Metrics & Regression. cekura.ai/blogs.
11. WebRTC.ventures. QA Testing for AI Voice Agents: A Real-Time Communication QA Framework. 2026.